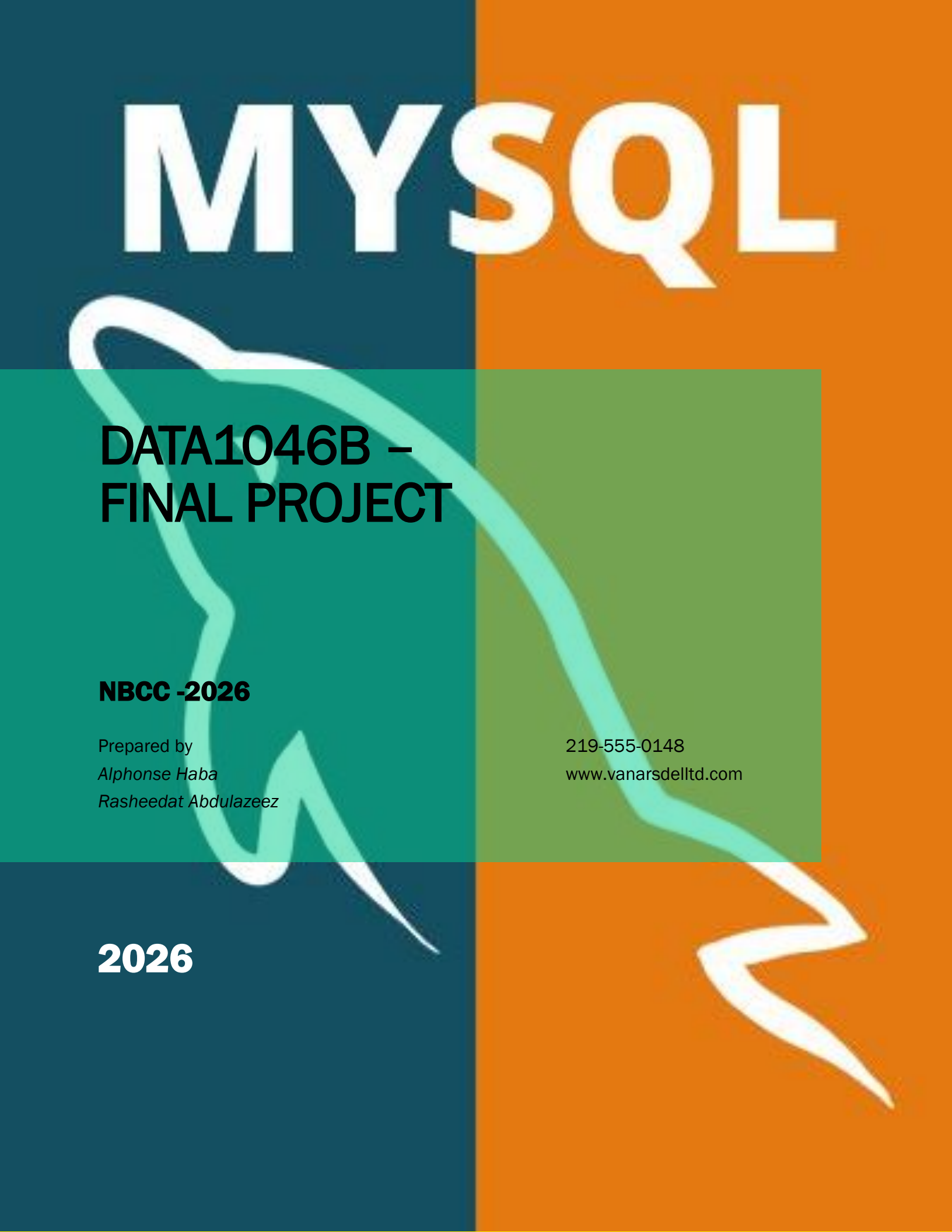


DATA1046B – FINAL PROJECT

NBCC -2026

Prepared by
Alphonse Haba
Rasheedat Abdulazeez

219-555-0148
www.vanarsdelltd.com

2026

FINAL PROJECT REPORT

INTRODUCTION:

AS defined, MySQL Workbench is a graphical tool used to design, develop, and manage MySQL databases. In this project, it was used to create the Hospital DBMS model, define relationships between tables, execute SQL statements, and examine query results.

DDL (Data Definition Language) refers to SQL commands that define or modify database structures. Examples include CREATE, ALTER, and DROP. In the Hospital DBMS, DDL was used to create tables, define primary and foreign keys, and create views.

DML (Data Manipulation Language) refers to SQL commands used to insert, modify, and remove records. Examples include INSERT, UPDATE, and DELETE. In this project, DML was used to populate hospital tables with sample records. SELECT retrieves data and is often classified separately as Data Query Language (DQL).

ERD (Entity–Relationship Diagram) is a visual representation of database entities, their attributes, and their relationships. The Hospital DBMS ERD shows how doctors, patients, appointments, specializations, and treatments are connected. It represents one-to-many relationships through foreign keys and many-to-many relationships through junction tables, such as Patient_has_Treatment and Treatment_has_Doctor.

PROJECT PROCESS BREAKDOWN:

As designed in the first lab, the final Entity–Relationship Diagram (ERD), based on the specified relationships, is shown below.

Relationship	Cardinality	Business rule
Doctor → Patient	One-to-many	A doctor can treat multiple patients; each patient must have exactly one assigned doctor.
Patient → Appointment	One-to-many	A patient can make multiple appointments; each appointment belongs to exactly one patient.
Doctor → Appointment	One-to-many	A doctor can have multiple appointments; each appointment is associated with exactly one doctor.
Specialization → Doctor	One-to-many	A specialization can have multiple doctors; each doctor has exactly one specialization.
Patient ↔ Treatment	Many-to-many	A patient can receive multiple treatments, and a treatment can be received by multiple patients.
Doctor ↔ Treatment	Many-to-many	A doctor can administer multiple treatments, and a treatment can be administered by multiple doctors.

The visual representation in the ER diagram is shown in the figure below.

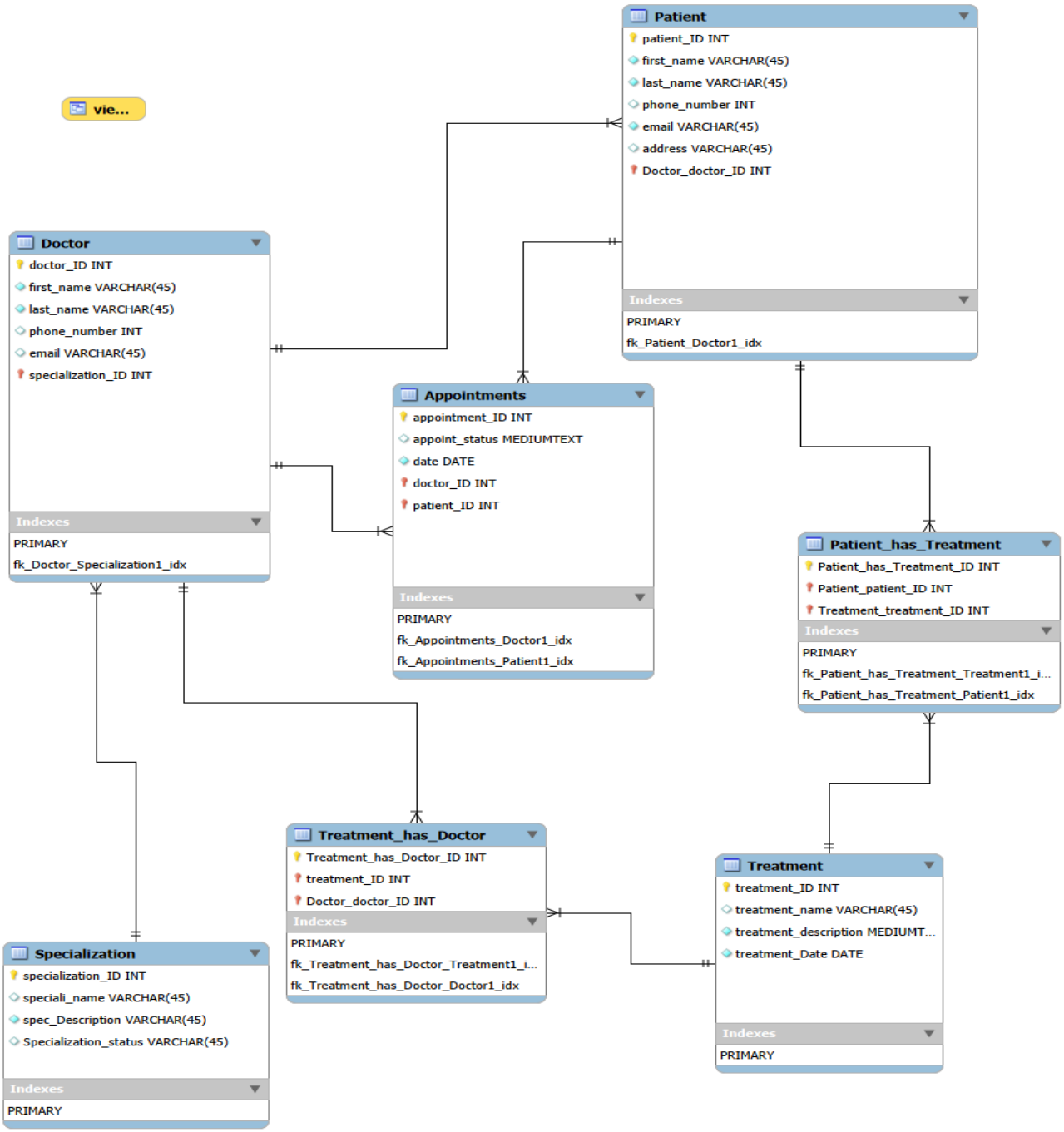


Figure 1 ER diagram of myhospital Databases

The transition from an Entity–Relationship Diagram (ERD) to a database can follow two approaches: manually writing CREATE TABLE statements to define tables, attributes, data types, and constraints, or using MySQL Workbench's forward engineering feature to generate and execute the SQL script from the model. In this project, we used forward engineering to implement the Hospital DBMS, as illustrated in the screenshots below.

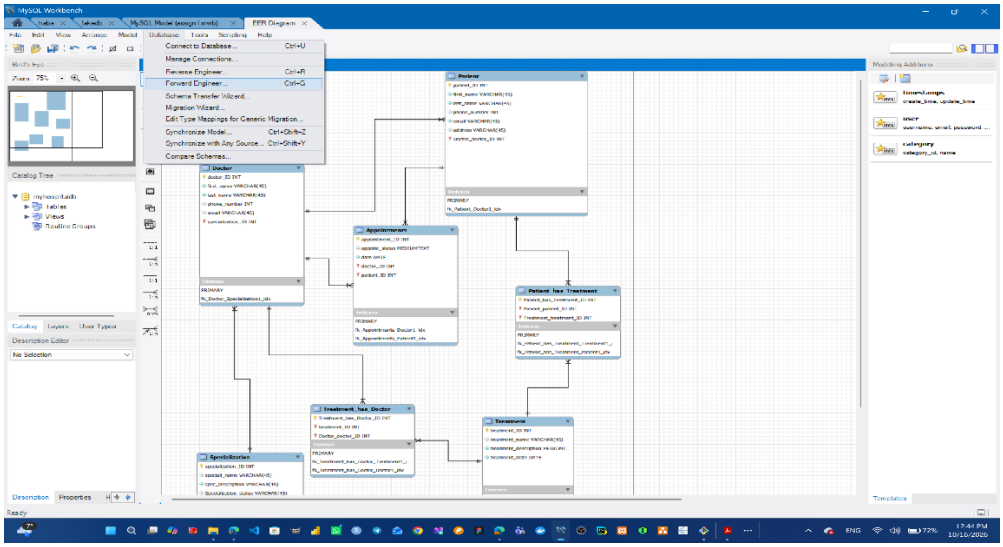
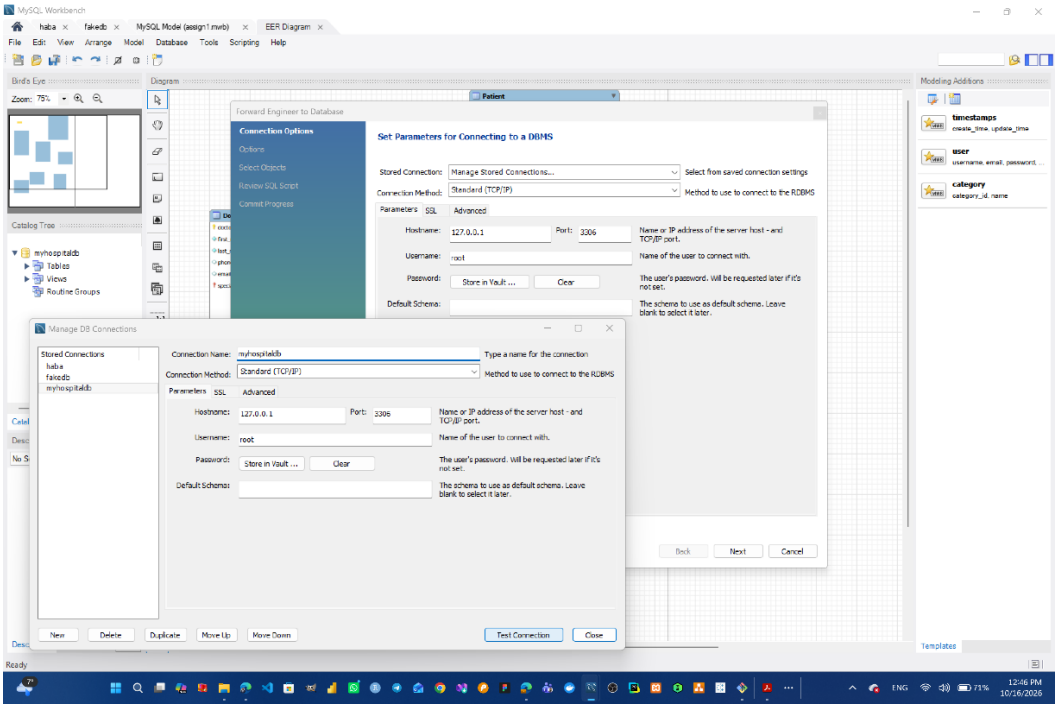
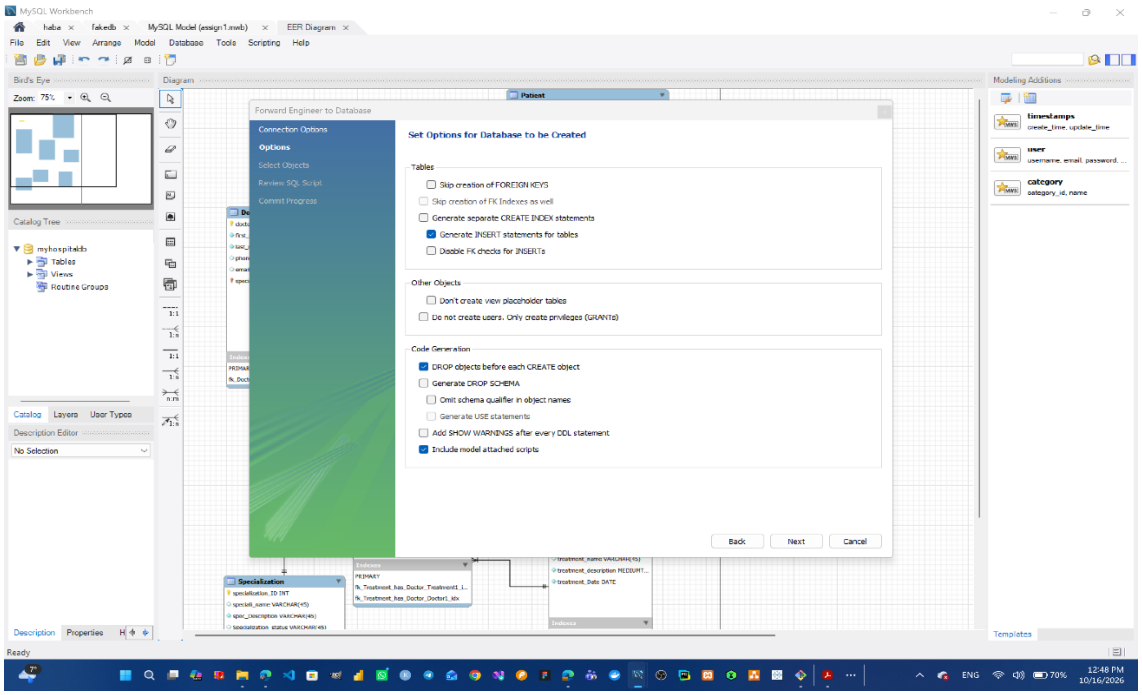


Figure 2-engineering forward

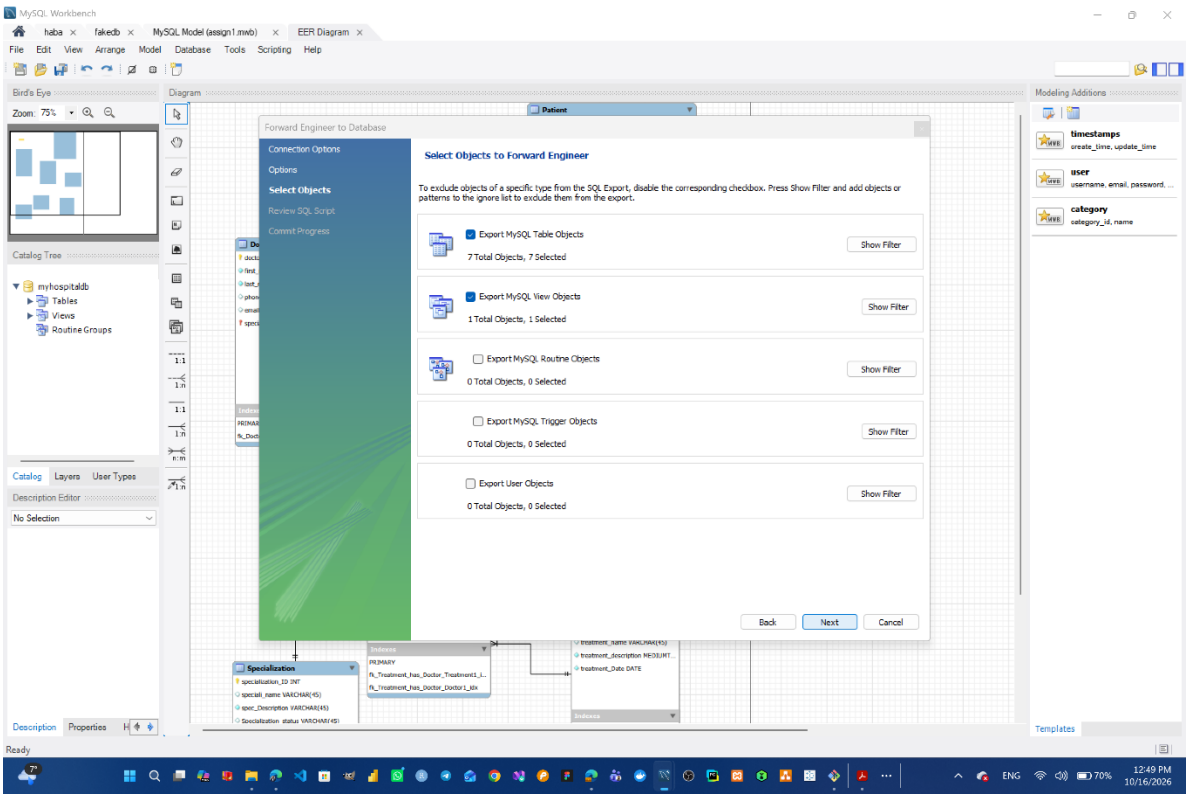
After opening the database and then choosing engineering forward, we create a new database and establish a new connection to our "myhospitaldb" using the step shown in the images below.



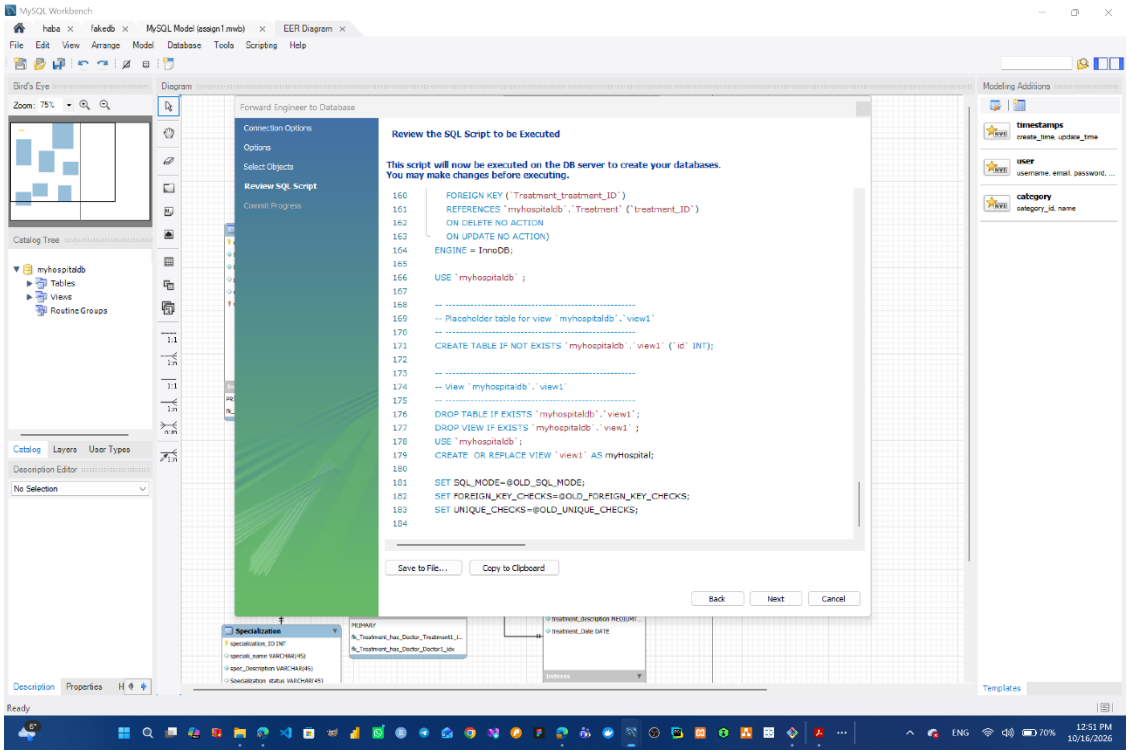
We configured the forward engineering options by enabling or disabling the inclusion of INSERT statements, depending on whether the generated SQL script should also populate the tables with data.



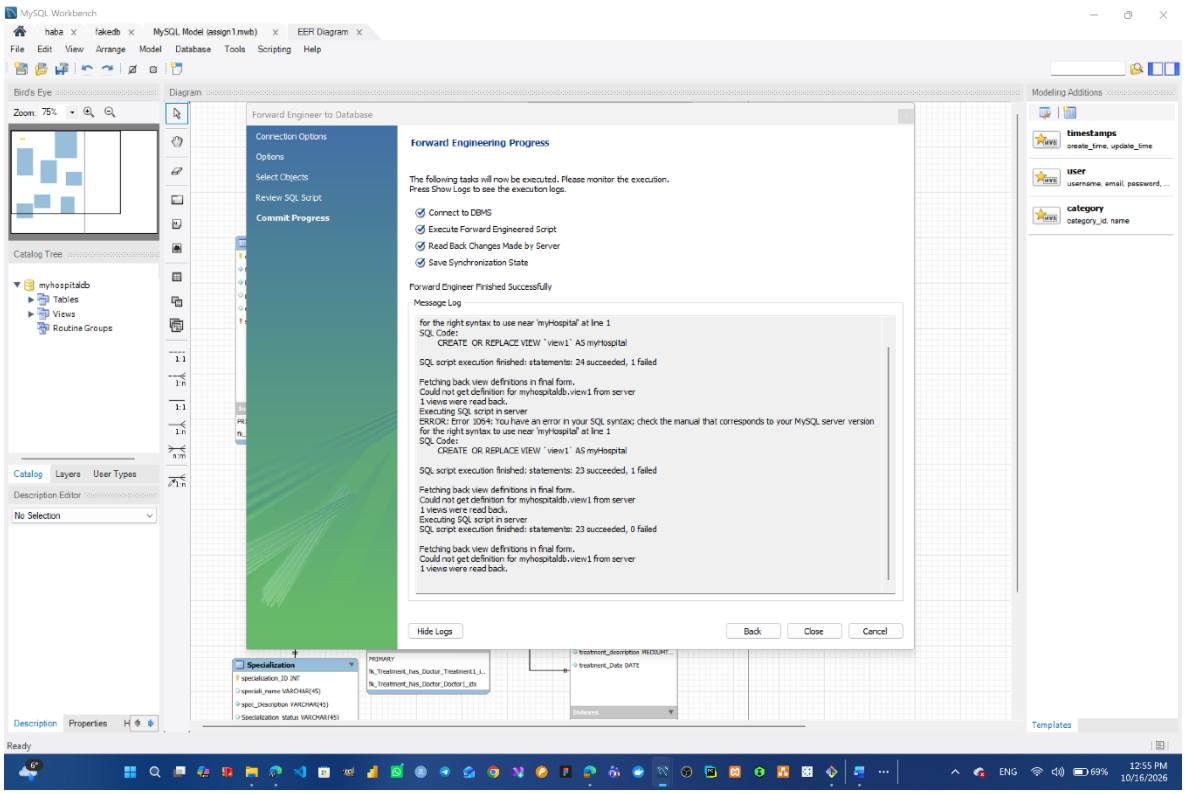
Then click on “next button”



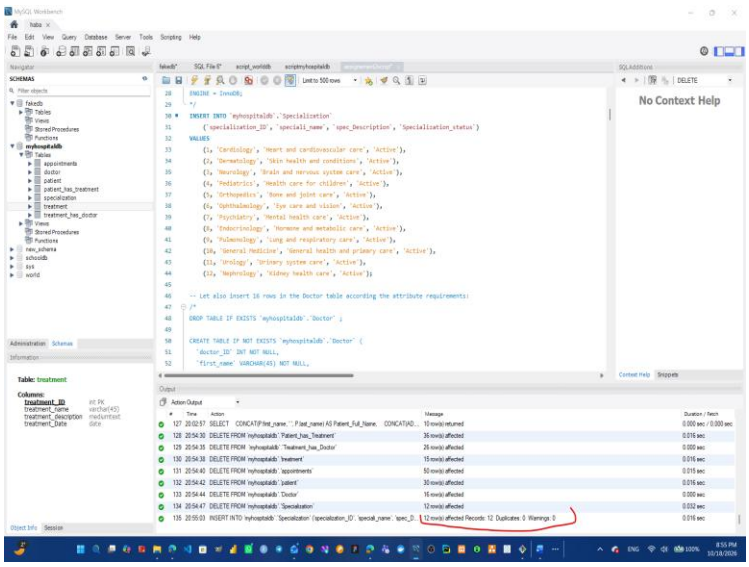
The script was automatically inserted, and our model was built successfully.

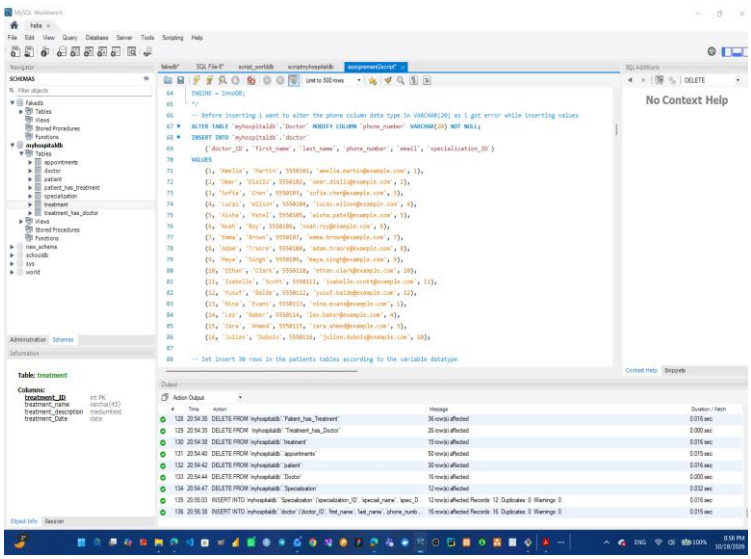


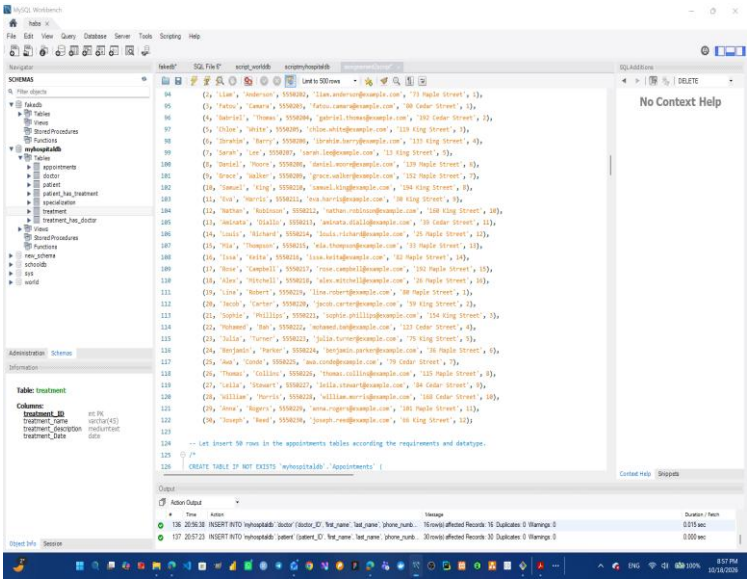
Finally, closed the wizard and refreshed the Schemas panel to display the newly created database and its tables.

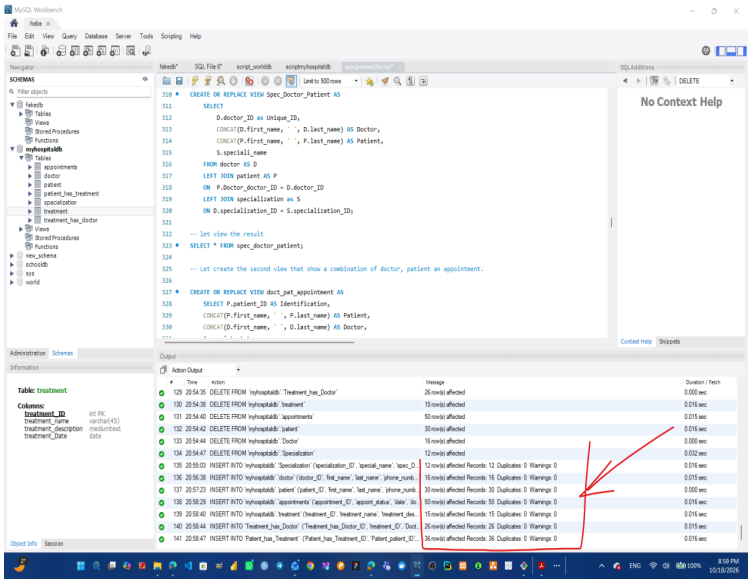


After creating the database, we populated the Hospital DBMS using SQL INSERT INTO statements, inserting 12 records into Specialization, 16 into doctor, 30 into patient, 50 into Appointments, 15 into Treatment, 26 into Treatment_has_Doctor, and 36 into patient_has_Treatment, for a total of 185 records. Each result is shown in the pictures below:

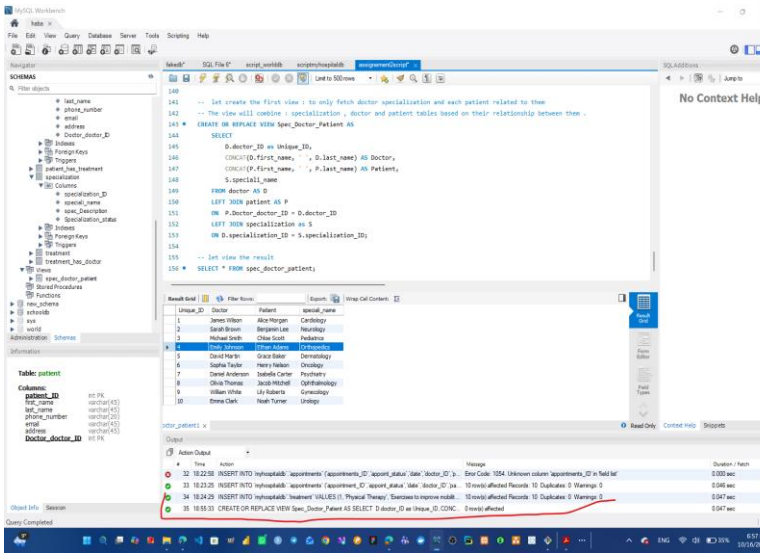


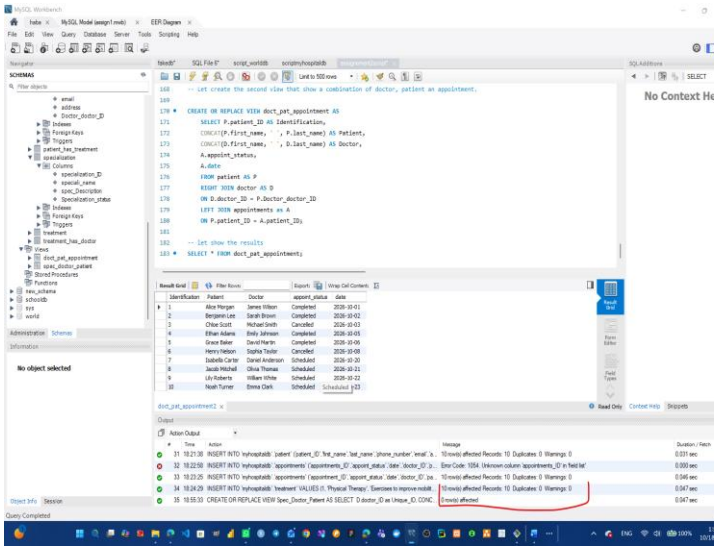


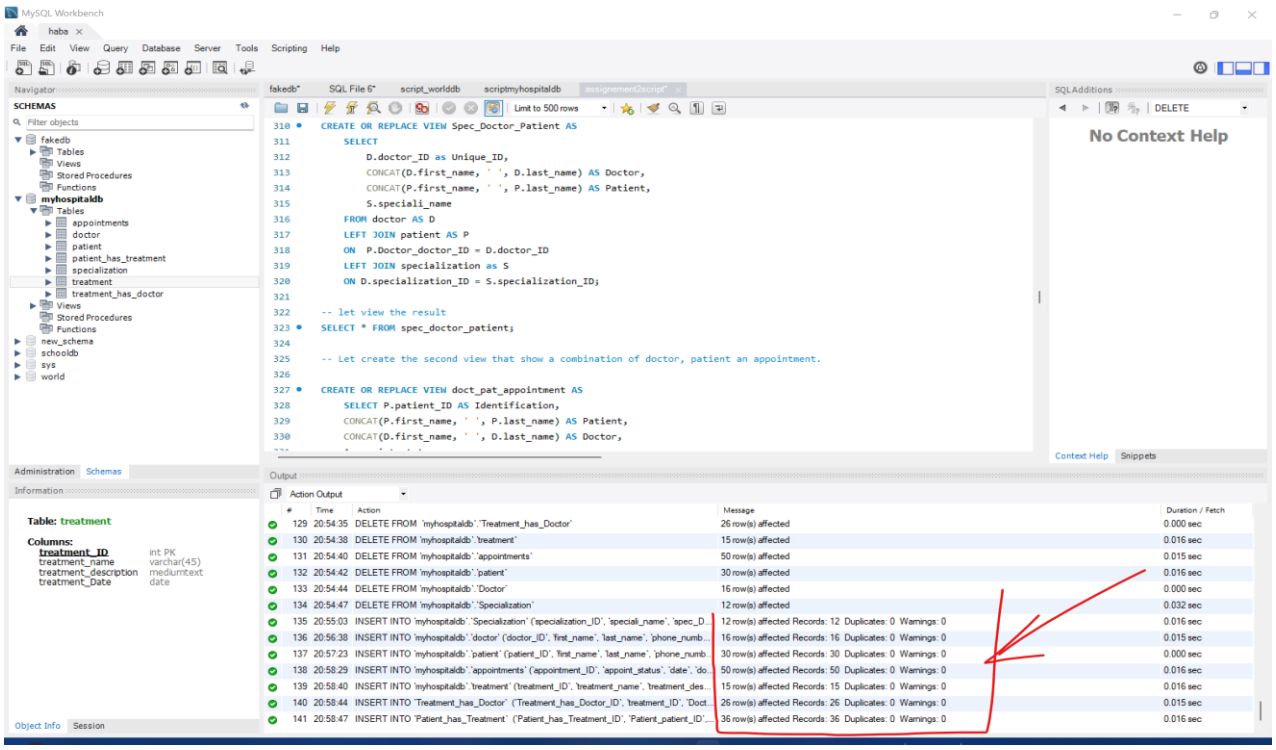




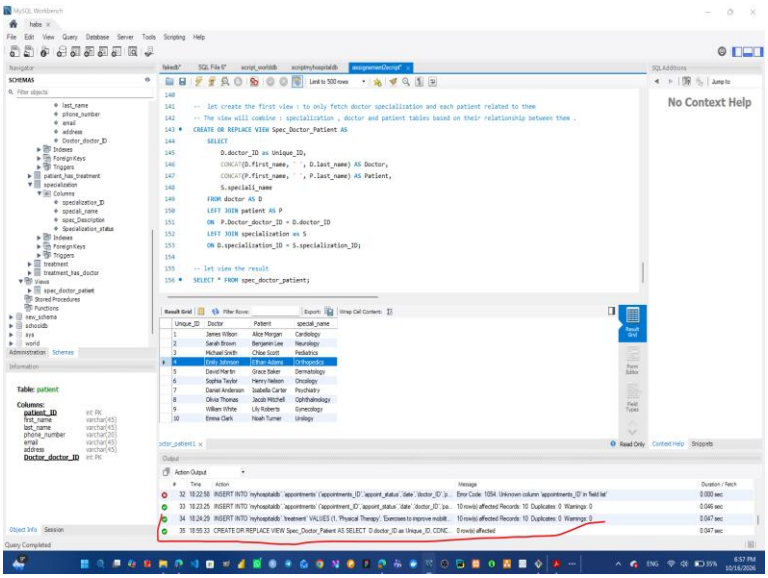
After successfully inserted the record in each table, we created three views using CREATE OR REPLACE VIEW to simplify access to related hospital data: Spec_Doctor_Patient combines doctors, their specializations, and assigned patients; doct_pat_appointment displays patients, their assigned doctors, and appointment details; and patient_info brings together patient contact information, assigned doctors, appointments, and treatment history. All the results of these operations can be found in the pictures below.

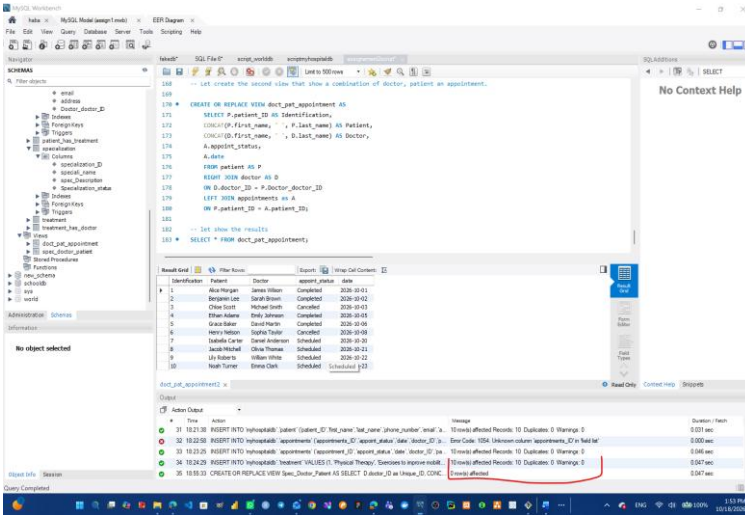


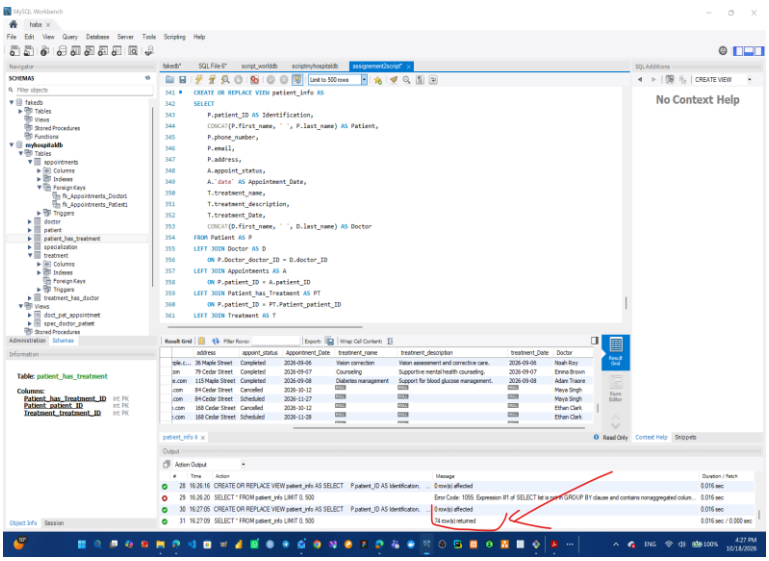


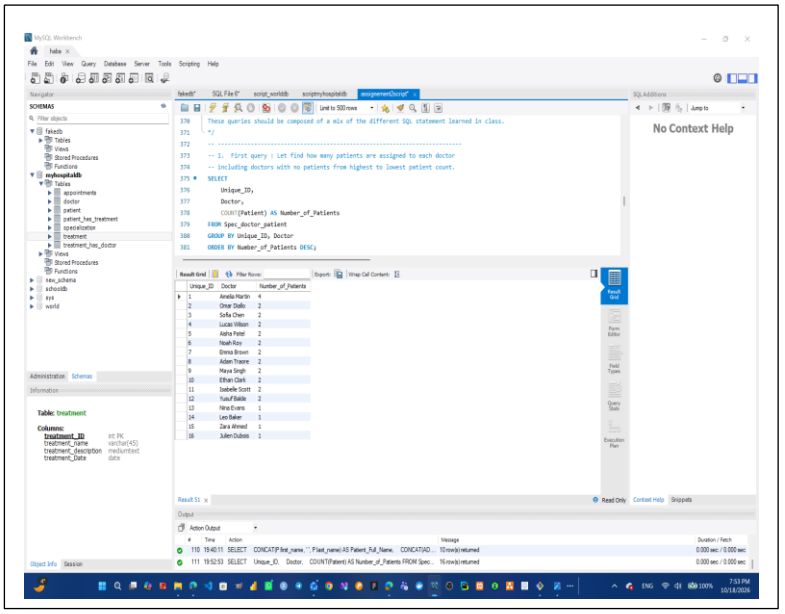


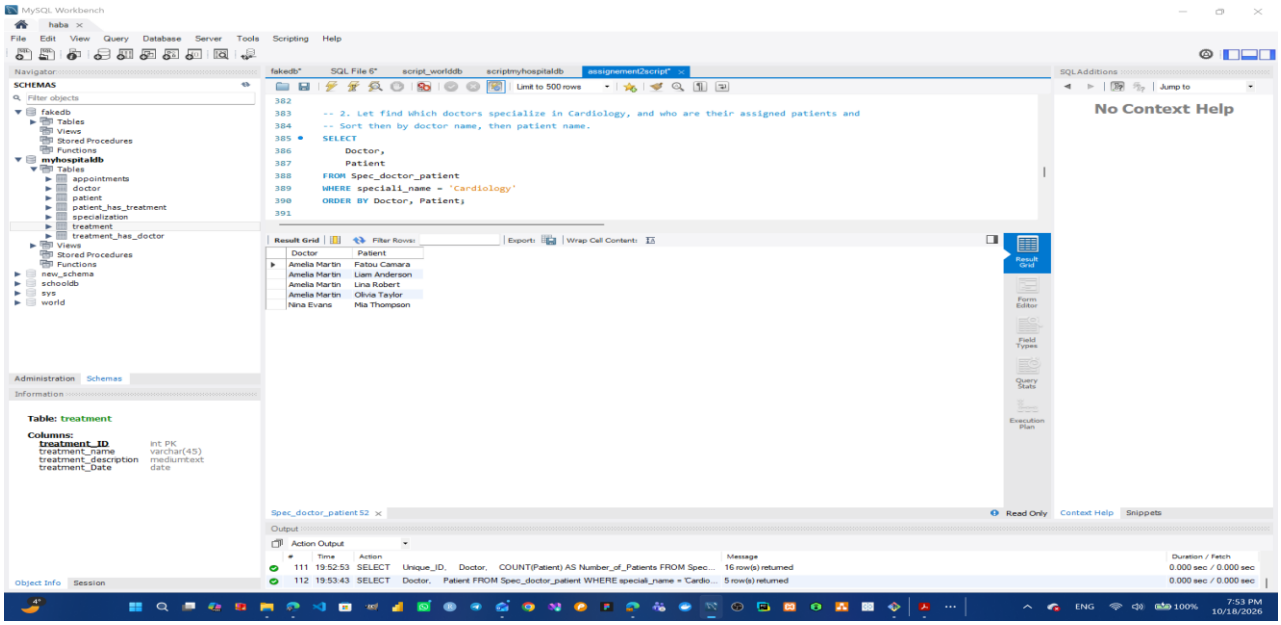
Successful completion of the above operations leads us to write two representative queries for each view to obtain meaningful information, including the number of patients per doctor, scheduled appointments, and treatment history. These queries combine filtering, sorting, grouping, and aggregation functions using SQL clauses such as WHERE, ORDER BY, and GROUP BY, as shown in the images below:

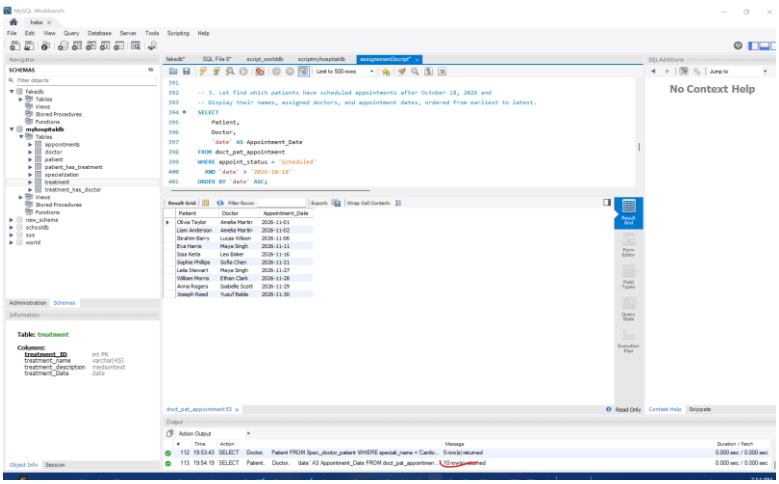


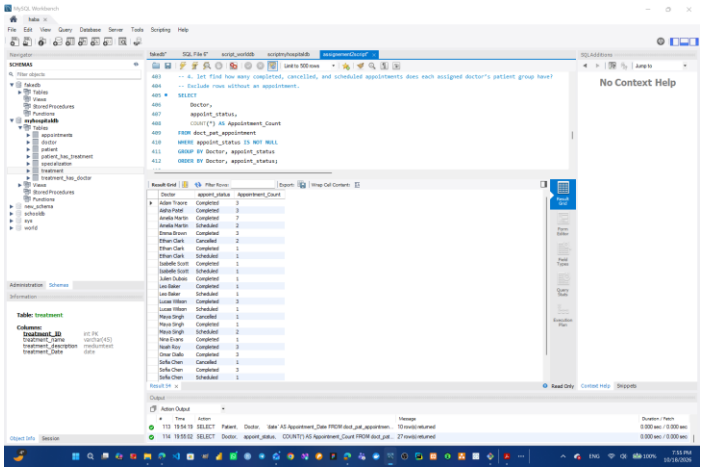


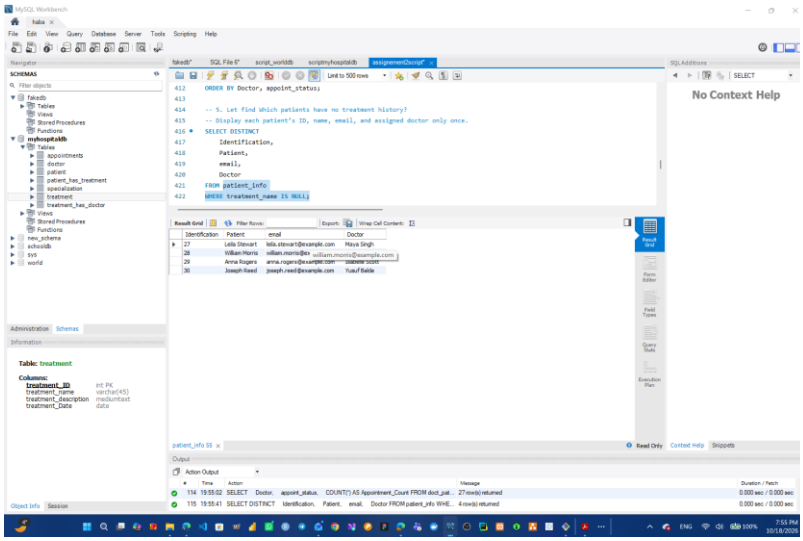


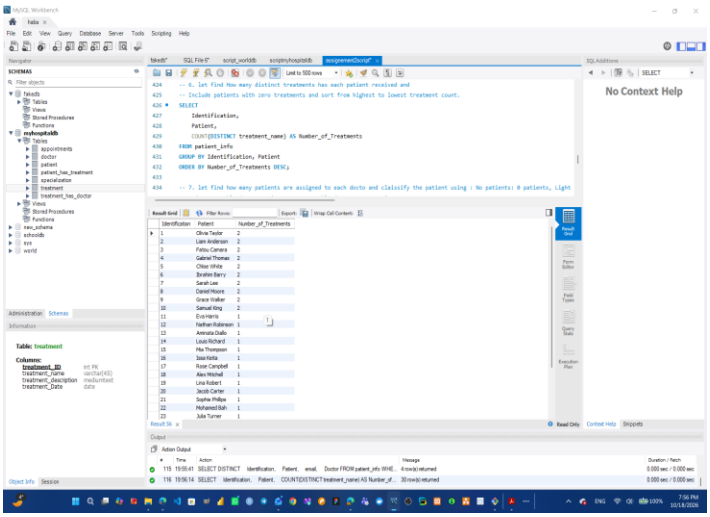



-- 2. Let find which doctors specialize in Cardiology, and who are their assigned patients and
-- Sort then by doctor name, then patient name.
SELECT
Doctor,
Patient
FROM Spec_doctor_patient
WHERE speciali_name = 'Cardiology'
ORDER BY Doctor, Patient;
No Context Help

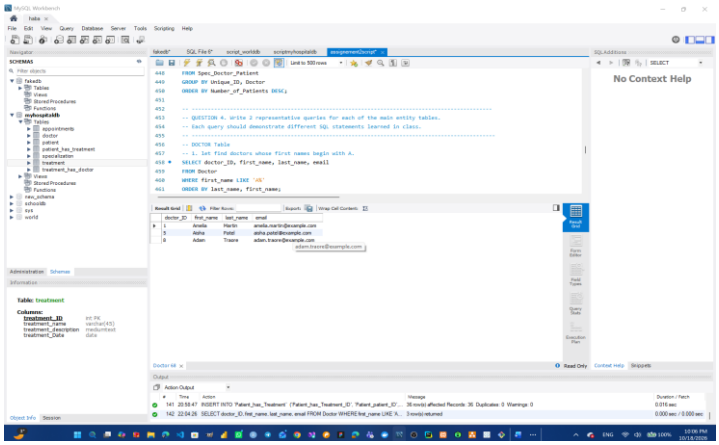

No Context Help

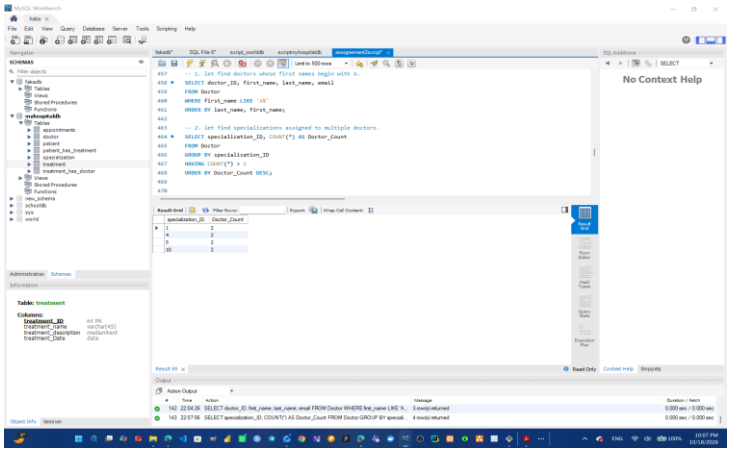

No Context Help

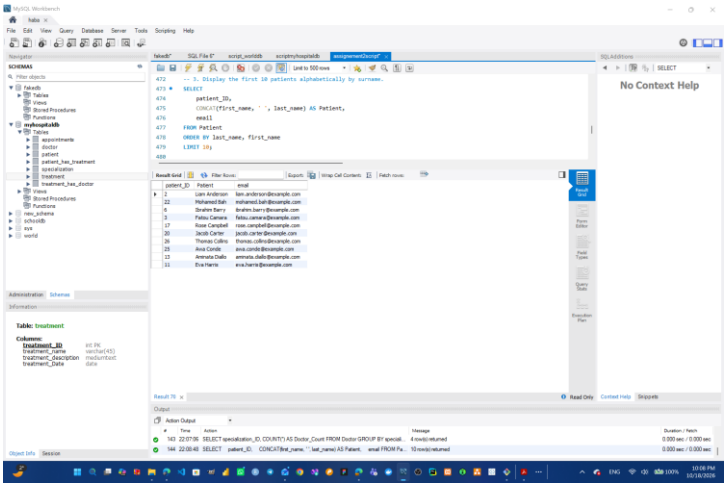

-- 5. Let find which patients have no treatment history?
-- Display each patient's ID, name, email, and assigned doctor only once.
SELECT DISTINCT
Identification,
Patient,
email,
Doctor
FROM patient_info
WHERE treatment_name IS NULL;
No Context Help

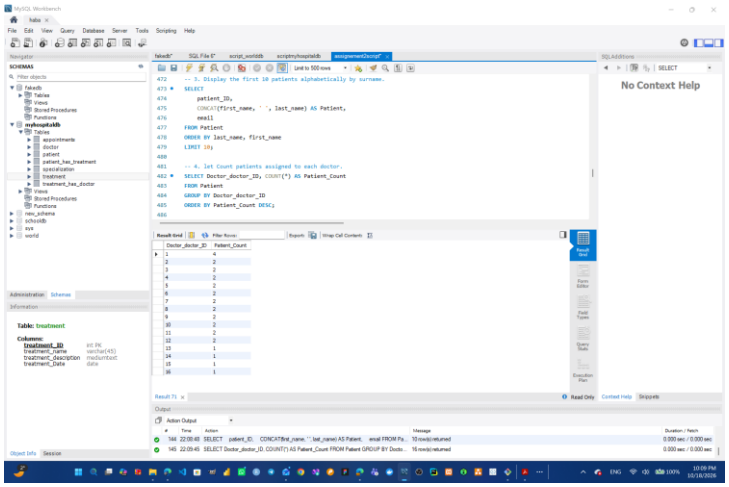

No Context Help

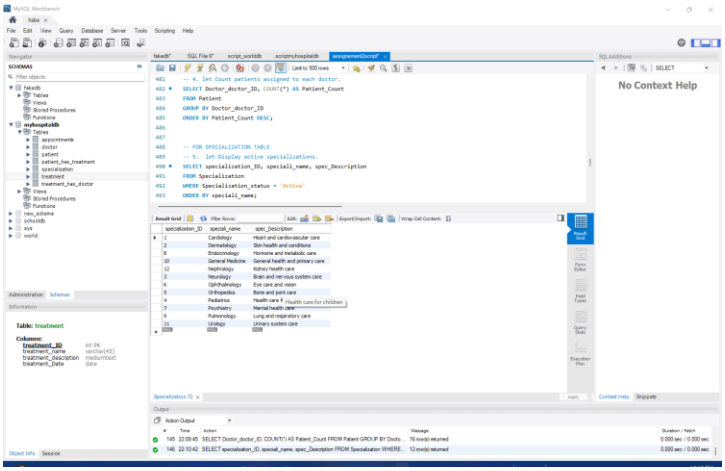
We wrote two representative queries for each of the five main entity tables to retrieve and summarize hospital data, demonstrating filtering, sorting, pattern matching, grouping, and aggregate functions through SQL statements such as WHERE, LIKE, ORDER BY, GROUP BY, and HAVING.

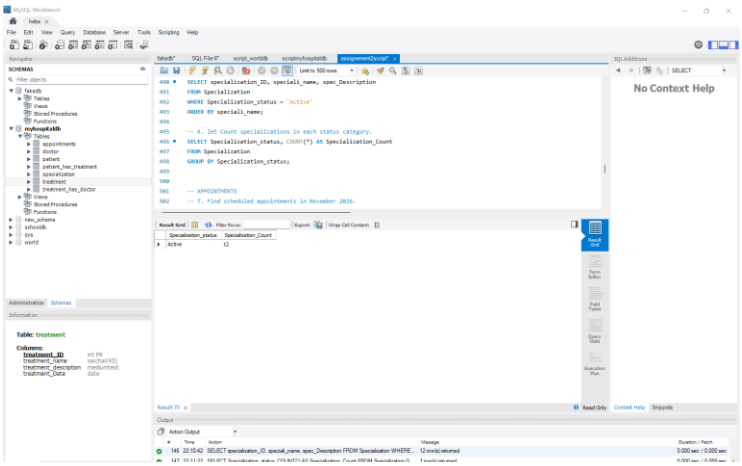


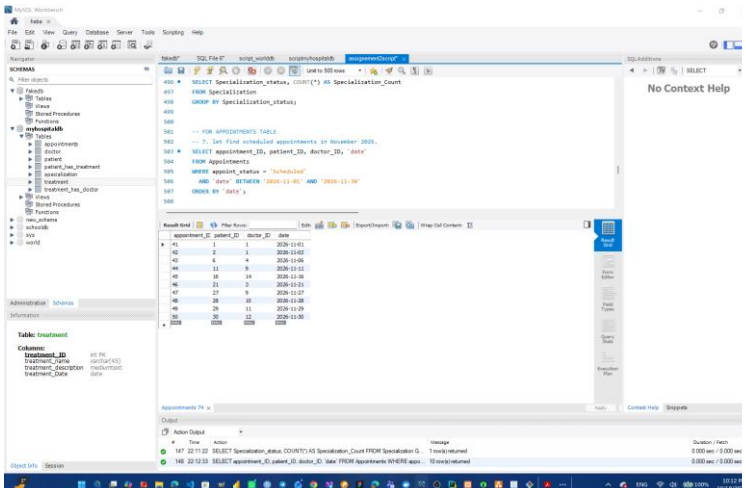


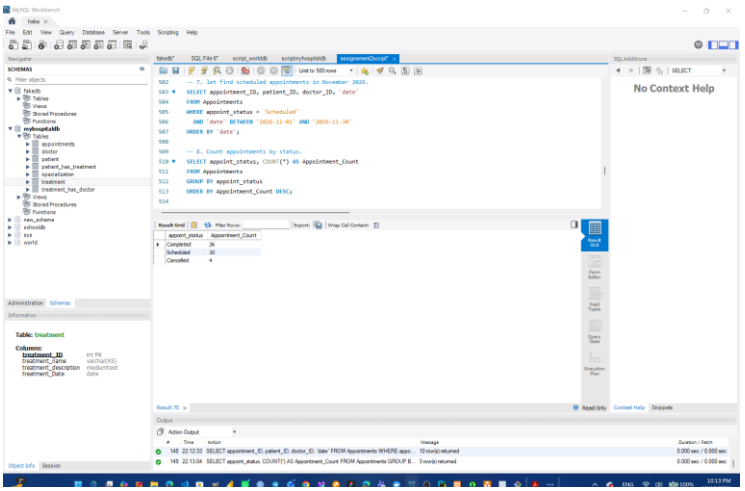


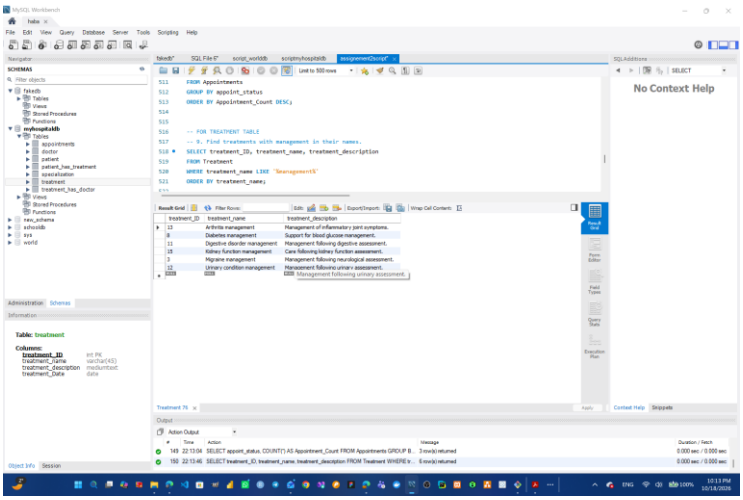


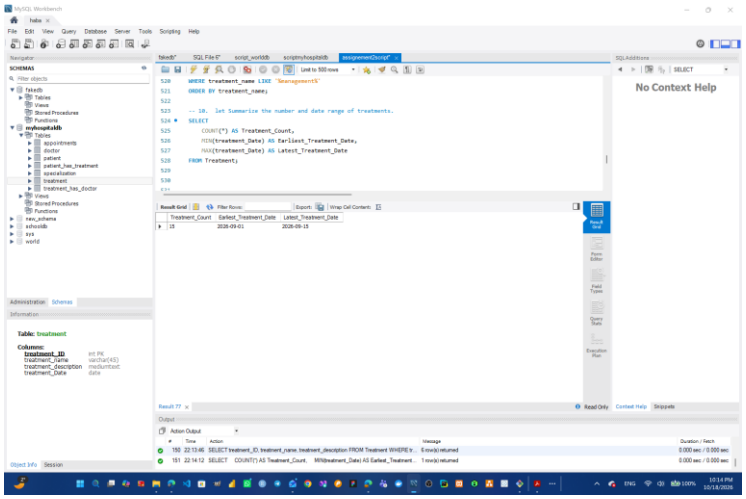












Finally, we wrote ten queries joining at least two tables, using INNER JOIN, LEFT JOIN, and CROSS JOIN to combine hospital data and retrieve meaningful information about patients, doctors, specializations, appointments, and treatments.

